

Assignment no 2

Parallel and Distributed Computing Lab - CS-426L

SUBMITTED BY:

Name: Shammas Ud Din

Student ID: 247154

Semester: 5th

Section: CYS-B

SUBMITTED TO:

Instructor Name: Mr. Qaiser Ali

Date of Submission: 25/10/26

AIR UNIVERSITY, KHARIAN CAMPUS

DEPARTMENT OF Computer Science

Question No. 1: Financial Trading Event Dispatcher Using Inheritance and Polymorphism

This task models a Complex Event Processing (CEP) engine for market events using four Java files. A superclass named TradeEvent holds the data every event shares, and two subclasses specialise its behaviour. The main application keeps all objects in TradeEvent references and calls dispatch() on each, showing that the JVM chooses the correct overridden method at run time.

File 1: TradeEvent.java

The superclass declares the protected fields senderId (String), timestamp (long) and basePrice (double), a constructor that sets them, and a baseline dispatch() method that prints the generic event details. Main concept: a single superclass holding the common state of every market event.

Java Program:

J TradeEvent.java

```java
public class TradeEvent {

    protected String senderId;
    protected long timestamp;
    protected double basePrice;

    public TradeEvent(String senderId, long timestamp, double basePrice) {
        this.senderId = senderId;
        this.timestamp = timestamp;
        this.basePrice = basePrice;
    }

    public void dispatch() {
        System.out.println("Generic Trade Event");
        System.out.println("Sender ID  : " + senderId);
        System.out.println("Timestamp  : " + timestamp);
        System.out.println("Base Price : " + basePrice);
    }
}
```

File 2: FIXOrderEvent.java

This subclass inherits the common fields, adds clOrdId and quantity, and calls super(...) in its constructor. Its overridden dispatch() prints a stock purchase request and adds a 0.15% exchange transaction fee to the order value. Main concept: inheritance with super() and method overriding.

Java Program:

FIXOrderEvent.java

```
public class FIXOrderEvent extends TradeEvent {

    private static final double EXCHANGE_FEE_RATE = 0.0015; // 0.15%

    private String clOrdId;
    private int quantity;

    public FIXOrderEvent(String senderId, long timestamp, double basePrice,
                         String clOrdId, int quantity) {
        super(senderId, timestamp, basePrice);
        this.clOrdId = clOrdId;
        this.quantity = quantity;
    }

    @Override
    public void dispatch() {
        double orderValue = basePrice * quantity;
        double fee = orderValue * EXCHANGE_FEE_RATE;
        double totalCost = orderValue + fee;

        System.out.println("FIX Stock Purchase Order");
        System.out.println("Sender ID        : " + senderId);
        System.out.println("Timestamp        : " + timestamp);
        System.out.println("ClOrdID          : " + clOrdId);
        System.out.println("Quantity         : " + quantity + " shares");
        System.out.printf("Price per share  : %.2f%n", basePrice);
        System.out.printf("Order value      : %.2f%n", orderValue);
        System.out.printf("Exchange fee 0.15%%: %.2f%n", fee);
        System.out.printf("Total cost       : %.2f%n", totalCost);
        System.out.println("Status           : ORDER EXECUTED");
    }
}
```

File 3: ReutersQuoteEvent.java

This subclass reuses the inherited fields and adds bidPrice and askPrice. Its dispatch() reports the bid-ask spread, the mid price and the spread percentage. Main concept: the same method call produces entirely different market analytics.

Java Program:

ReutersQuoteEvent.java

```
public class ReutersQuoteEvent extends TradeEvent {

    private double bidPrice;
```

```
4         private double askPrice;
5
6         public ReutersQuoteEvent(String senderId, long timestamp, double basePrice,
7                                  double bidPrice, double askPrice) {
8             super(senderId, timestamp, basePrice);
9             this.bidPrice = bidPrice;
10            this.askPrice = askPrice;
11        }
12
13        @Override
14        public void dispatch() {
15            double spread = askPrice - bidPrice;
16            double midPrice = (askPrice + bidPrice) / 2.0;
17            double spreadPercent = (spread / midPrice) * 100.0;
18
19            System.out.println("Reuters Real-Time Quote");
20            System.out.println("Sender ID     : " + senderId);
21            System.out.println("Timestamp     : " + timestamp);
22            System.out.printf("Reference     : %.2f%n", basePrice);
23            System.out.printf("Bid Price     : %.2f%n", bidPrice);
24            System.out.printf("Ask Price     : %.2f%n", askPrice);
25            System.out.printf("Bid-Ask Spread: %.2f%n", spread);
26            System.out.printf("Mid Price     : %.3f%n", midPrice);
27            System.out.printf("Spread %%      : %.4f%%%n", spreadPercent);
28        }
29    }
```

File 4: TradingEngineApp.java

The driver creates one FIXOrderEvent, one ReutersQuoteEvent and one plain TradeEvent, stores all three in TradeEvent-typed variables and loops over them calling dispatch(). Main concept: runtime polymorphism, where the actual object type, not the reference type, decides which method runs.

Java Program:

J TradingEngineApp.java

```
1     public class TradingEngineApp {
2
3         public static void main(String[] args) {
4
5             // Subclass objects stored in SUPERCLASS reference variables
6             TradeEvent event1 = new FIXOrderEvent(
7                     "FIX-GATEWAY-01", 1761300000000L, 185.50, "ORD-90001", 100);
8
9             TradeEvent event2 = new ReutersQuoteEvent(
10                    "REUTERS-FEED-07", 1761300000450L, 185.55, 185.48, 185.62);
11
12            TradeEvent event3 = new TradeEvent(
13                    "GENERIC-SOURCE", 1761300000900L, 100.00);
```

```
14
15          TradeEvent[] events = { event1, event2, event3 };
16
17          System.out.println("========================================");
18          System.out.println("   COMPLEX EVENT PROCESSING (CEP) ENGINE");
19          System.out.println("========================================");
20
21          for (TradeEvent event : events) {
22              // Runtime polymorphism: JVM picks the overridden dispatch()
23              event.dispatch();
24              System.out.println("----------------------------------------");
25          }
26
27          System.out.println("      ALL EVENTS DISPATCHED SUCCESSFULLY");
28          System.out.println("========================================");
29      }
30  }
```

Program Output:

```
> java TradingEngineApp
========================================
   COMPLEX EVENT PROCESSING (CEP) ENGINE
========================================
FIX Stock Purchase Order
Sender ID        : FIX-GATEWAY-01
Timestamp        : 1761300000000
ClOrdID          : ORD-90001
Quantity         : 100 shares
Price per share  : 185.50
Order value      : 18550.00
Exchange fee 0.15%: 27.83
Total cost       : 18577.83
Status           : ORDER EXECUTED
----------------------------------------
Reuters Real-Time Quote
Sender ID     : REUTERS-FEED-07
Timestamp     : 1761300000450
Reference     : 185.55
Bid Price     : 185.48
Ask Price     : 185.62
Bid-Ask Spread: 0.14
Mid Price     : 185.550
Spread %      : 0.0755%
----------------------------------------
Generic Trade Event
Sender ID  : GENERIC-SOURCE
Timestamp  : 1761300000900
Base Price : 100.0
----------------------------------------
      ALL EVENTS DISPATCHED SUCCESSFULLY
========================================
```

Explanation

For the FIX order, the order value is 185.50 × 100 = 18,550.00. The 0.15% fee is 18,550.00 × 0.0015 = 27.825, shown as 27.83, so the total cost is 18,577.83. For the Reuters quote, the spread is the ask minus the bid: 185.62 − 185.48 = 0.14.

Question No. 2: Protocol Packet Verification Using a Custom Stack and Queue

Here an array-backed stack and an array-backed queue are built from scratch, without any built-in collection classes. The auditor feeds each character of a packet into both structures, then pops and dequeues together. A stack returns characters in reverse (LIFO) while a queue returns them in the original order (FIFO), so the packet is a palindrome only when every pair matches.

File 1: TransactionStack.java

A fixed-capacity char array with a top pointer that starts at -1. It provides push, pop, peek and isEmpty, with overflow and underflow checks. Main concept: LIFO, where the last character pushed is the first popped.

Java Program:

J TransactionStack.java

```java
public class TransactionStack {

    private char[] stack;
    private int top;
    private int capacity;

    public TransactionStack(int capacity) {
        this.capacity = capacity;
        stack = new char[capacity];
        top = -1;
    }

    public void push(char c) {
        if (top == capacity - 1) {
            System.out.println("Stack overflow!");
            return;
        }
        stack[++top] = c;
    }

    public char pop() {
        if (isEmpty()) {
            System.out.println("Stack underflow!");
            return '\0';
        }
        return stack[top--];
    }

    public char peek() {
        if (isEmpty()) {
            return '\0';
        }
```

```
            return stack[top];
        }

        public boolean isEmpty() {
            return top == -1;
        }
    }
```

File 2: RequestQueue.java

A circular char array using front, rear and size so that freed slots are reused. It offers enqueue, dequeue and isEmpty with overflow and underflow checks. Main concept: FIFO, where the first character enqueued is the first dequeued.

Java Program:

J RequestQueue.java

```
public class RequestQueue {

    private char[] queue;
    private int front;
    private int rear;
    private int size;
    private int capacity;

    public RequestQueue(int capacity) {
        this.capacity = capacity;
        queue = new char[capacity];
        front = 0;
        rear = -1;
        size = 0;
    }

    public void enqueue(char c) {
        if (size == capacity) {
            System.out.println("Queue overflow!");
            return;
        }
        rear = (rear + 1) % capacity;
        queue[rear] = c;
        size++;
    }

    public char dequeue() {
        if (isEmpty()) {
            System.out.println("Queue underflow!");
            return '\0';
        }
        char c = queue[front];
        front = (front + 1) % capacity;
        size--;
        return c;
```

```
    }

    public boolean isEmpty() {
        return size == 0;
    }
}
```

File 3: PacketSymmetryAuditor.java

verifySymmetry() loads the payload into both structures, then compares each popped and dequeued pair and reports MATCH or MISMATCH. The main method tests several sample packets. Main concept: comparing a reversed stream with the original stream reveals a palindrome signature.

Java Program:

PacketSymmetryAuditor.java

```
public class PacketSymmetryAuditor {

    // Returns true when the payload reads the same forwards and backwards
    public static boolean verifySymmetry(String payload) {

        TransactionStack stack = new TransactionStack(payload.length());
        RequestQueue queue = new RequestQueue(payload.length());

        System.out.println("Ingesting packet : " + payload);

        // Push into the LIFO stack while enqueueing into the FIFO queue
        for (int i = 0; i < payload.length(); i++) {
            char c = payload.charAt(i);
            stack.push(c);
            queue.enqueue(c);
        }

        boolean symmetric = true;
        int position = 1;

        // Pop (reverse order) and dequeue (original order) together
        while (!stack.isEmpty() && !queue.isEmpty()) {
            char fromStack = stack.pop();
            char fromQueue = queue.dequeue();

            String result = (fromStack == fromQueue) ? "MATCH" : "MISMATCH";
            System.out.println("  Step " + position + ": pop = '" + fromStack
                    + "'  dequeue = '" + fromQueue + "'  -> " + result);

            if (fromStack != fromQueue) {
                symmetric = false;
            }
            position++;
```

```
            }
            return symmetric;
        }

        public static void main(String[] args) {

            String[] packets = { "A1B2B1A", "level", "X9Y8Z7", "racecar" };

            System.out.println("========================================");
            System.out.println("      PACKET SYMMETRY VERIFICATION LOG");
            System.out.println("========================================");

            for (String packet : packets) {
                boolean symmetric = verifySymmetry(packet);
                System.out.println("Result : " + packet + " -> "
                        + (symmetric ? "SYMMETRIC (Palindrome signature detected)"
                                     : "NOT SYMMETRIC"));
                System.out.println("----------------------------------------");
            }

            System.out.println("          AUDIT COMPLETE");
            System.out.println("========================================");
        }
    }
```

Program Output:

```
> java PacketSymmetryAuditor
========================================
      PACKET SYMMETRY VERIFICATION LOG
========================================
Ingesting packet : A1B2B1A
  Step 1: pop = 'A'  dequeue = 'A'  -> MATCH
  Step 2: pop = '1'  dequeue = '1'  -> MATCH
  Step 3: pop = 'B'  dequeue = 'B'  -> MATCH
  Step 4: pop = '2'  dequeue = '2'  -> MATCH
  Step 5: pop = 'B'  dequeue = 'B'  -> MATCH
  Step 6: pop = '1'  dequeue = '1'  -> MATCH
  Step 7: pop = 'A'  dequeue = 'A'  -> MATCH
Result : A1B2B1A -> SYMMETRIC (Palindrome signature detected)
----------------------------------------
Ingesting packet : level
  Step 1: pop = 'l'  dequeue = 'l'  -> MATCH
  Step 2: pop = 'e'  dequeue = 'e'  -> MATCH
  Step 3: pop = 'v'  dequeue = 'v'  -> MATCH
  Step 4: pop = 'e'  dequeue = 'e'  -> MATCH
  Step 5: pop = 'l'  dequeue = 'l'  -> MATCH
Result : level -> SYMMETRIC (Palindrome signature detected)
----------------------------------------
Ingesting packet : X9Y8Z7
  Step 1: pop = '7'  dequeue = 'X'  -> MISMATCH
  Step 2: pop = 'Z'  dequeue = '9'  -> MISMATCH
  Step 3: pop = '8'  dequeue = 'Y'  -> MISMATCH
```

```
  Step 4: pop = 'Y'  dequeue = '8'  -> MISMATCH
  Step 5: pop = '9'  dequeue = 'Z'  -> MISMATCH
  Step 6: pop = 'X'  dequeue = '7'  -> MISMATCH
Result : X9Y8Z7 -> NOT SYMMETRIC
----------------------------------------
Ingesting packet : racecar
  Step 1: pop = 'r'  dequeue = 'r'  -> MATCH
  Step 2: pop = 'a'  dequeue = 'a'  -> MATCH
  Step 3: pop = 'c'  dequeue = 'c'  -> MATCH
  Step 4: pop = 'e'  dequeue = 'e'  -> MATCH
  Step 5: pop = 'c'  dequeue = 'c'  -> MATCH
  Step 6: pop = 'a'  dequeue = 'a'  -> MATCH
  Step 7: pop = 'r'  dequeue = 'r'  -> MATCH
Result : racecar -> SYMMETRIC (Palindrome signature detected)
----------------------------------------
          AUDIT COMPLETE
========================================
```

Explanation

For "A1B2B1A" every popped character equals the dequeued one, so the packet is symmetric. For "X9Y8Z7" the very first pair, 7 against X, already differs, so the packet is not symmetric.

Question No. 3: Multithreaded Telemetry Engine Using Runnable and Thread

This program shows the two standard ways of creating threads in Java. One worker is defined by implementing Runnable, the other by extending Thread. The main class names both workers, starts them together and waits for them to finish, so their output lines interleave.

File 1: LogProcessorTask.java

Implements Runnable, which keeps the task separate from the thread that executes it. Its run() method prints five log iterations with the current thread name and sleeps 1000 ms between them.

Java Program:

LogProcessorTask.java

```java
public class LogProcessorTask implements Runnable {

    @Override
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println(
                Thread.currentThread().getName()
                + " - Log iteration: " + i
            );

            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return;
            }
        }
    }
}
```

File 2: TelemetryThread.java

Extends Thread and overrides run() directly. It prints five health updates showing CPU core count and used memory, pausing 800 ms between updates.

Java Program:

TelemetryThread.java

```java
public class TelemetryThread extends Thread {

```

```
        @Override
        public void run() {
            Runtime runtime = Runtime.getRuntime();

            for (int i = 1; i <= 5; i++) {
                long usedMemoryMB = (runtime.totalMemory() - runtime.freeMemory())
                                    / (1024 * 1024);
                int cpuCores = runtime.availableProcessors();

                System.out.println(
                    Thread.currentThread().getName()
                    + " - System health update: " + i
                    + " | CPU cores: " + cpuCores
                    + " | Memory used: " + usedMemoryMB + " MB"
                );

                try {
                    Thread.sleep(800);
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                    return;
                }
            }
        }
    }
```

File 3: AsyncWebEngineApp.java

Prints the main thread name, creates both workers, names them "Telemetry-Worker" and "Log-Worker", shows their states before start(), starts them, reports isAlive() and uses join() so main waits for both. Main concept: concurrency.

Java Program:

J AsyncWebEngineApp.java

```
    public class AsyncWebEngineApp {

        public static void main(String[] args) throws InterruptedException {

            System.out.println("Main thread: " + Thread.currentThread().getName());

            // Thread created by extending Thread
            TelemetryThread telemetry = new TelemetryThread();
            telemetry.setName("Telemetry-Worker");

            // Thread created by passing a Runnable into a new Thread object
            LogProcessorTask logTask = new LogProcessorTask();
            Thread logThread = new Thread(logTask);
            logThread.setName("Log-Worker");
```

```
15
16          System.out.println("Telemetry-Worker state before start(): " + telemetry.getState());
17          System.out.println("Log-Worker state before start()      : " + logThread.getState());
18
19          // Start both threads so they run concurrently
20          telemetry.start();
21          logThread.start();
22
23          System.out.println("Active threads started.");
24          System.out.println("Telemetry-Worker alive: " + telemetry.isAlive());
25          System.out.println("Log-Worker alive      : " + logThread.isAlive());
26
27          // Main waits for both workers to finish before terminating
28          telemetry.join();
29          logThread.join();
30
31          System.out.println("Both worker threads completed. Main thread exiting.");
32      }
33  }
```

Program Output:

```
> java AsyncWebEngineApp
Main thread: main
Telemetry-Worker state before start(): NEW
Log-Worker state before start()      : NEW
Active threads started.
Log-Worker - Log iteration: 1
Telemetry-Worker alive: true
Log-Worker alive      : true
Telemetry-Worker - System health update: 1 | CPU cores: 1 | Memory used: 2 MB
Telemetry-Worker - System health update: 2 | CPU cores: 1 | Memory used: 3 MB
Log-Worker - Log iteration: 2
Telemetry-Worker - System health update: 3 | CPU cores: 1 | Memory used: 3 MB
Log-Worker - Log iteration: 3
Telemetry-Worker - System health update: 4 | CPU cores: 1 | Memory used: 3 MB
Log-Worker - Log iteration: 4
Telemetry-Worker - System health update: 5 | CPU cores: 1 | Memory used: 3 MB
Log-Worker - Log iteration: 5
Both worker threads completed. Main thread exiting.
```

Explanation

Calling start() creates a new thread of execution, whereas calling run() directly would execute on the main thread. Because the sleep intervals (800 ms and 1000 ms) differ, the telemetry and log lines interleave. The exact order can vary slightly between runs because the operating system schedules the threads.

Question No. 4: Parallel Portfolio Assessment Engine Using Fork/Join

This task uses the Fork/Join framework to assess a large portfolio of asset values. One task sums the risk scores, and another applies a stress-test penalty in place. The driver runs both in a ForkJoinPool and compares timings with serial computation.

File 1: ParallelRiskCalculator.java

Extends RecursiveTask<Double> and returns the total risk score. A segment of at most 1000 elements is summed directly; a larger one is split in two, one half is forked, the other computed in the current thread, and the results are joined. Main concept: divide and conquer.

Java Program:

ParallelRiskCalculator.java

```java
import java.util.concurrent.RecursiveTask;

public class ParallelRiskCalculator extends RecursiveTask<Double> {

    private static final int THRESHOLD = 1000;

    private final double[] assetValues;
    private final int start;
    private final int end;

    public ParallelRiskCalculator(double[] assetValues, int start, int end) {
        this.assetValues = assetValues;
        this.start = start;
        this.end = end;
    }

    @Override
    protected Double compute() {
        int length = end - start;

        // Small segment: compute the sum sequentially
        if (length <= THRESHOLD) {
            double sum = 0;
            for (int i = start; i < end; i++) {
                sum += assetValues[i];
            }
            return sum;
        }

        // Large segment: split into two sub-tasks
```

```
        int mid = start + length / 2;
        ParallelRiskCalculator left  = new ParallelRiskCalculator(assetValues, start, mid);
        ParallelRiskCalculator right = new ParallelRiskCalculator(assetValues, mid, end);

        left.fork();                        // run left asynchronously
        double rightResult = right.compute(); // compute right in this thread
        double leftResult = left.join();   // wait for left result

        return leftResult + rightResult;
    }
}
```

File 2: PortfolioThresholdTask.java

Extends RecursiveAction, returns nothing and edits the array in place. Every value above the risk cap is multiplied by 1.05 (a 5% volatility penalty), and large ranges are split recursively.

Java Program:

J PortfolioThresholdTask.java

```
import java.util.concurrent.RecursiveAction;

public class PortfolioThresholdTask extends RecursiveAction {

    private static final int THRESHOLD = 1000;
    private static final double STRESS_PENALTY = 1.05; // +5% volatility penalty

    private final double[] assetValues;
    private final int start;
    private final int end;
    private final double riskCap;

    public PortfolioThresholdTask(double[] assetValues, int start, int end, double riskCap) {
        this.assetValues = assetValues;
        this.start = start;
        this.end = end;
        this.riskCap = riskCap;
    }

    @Override
    protected void compute() {
        int length = end - start;

        if (length <= THRESHOLD) {
            // In-place adjustment of every value that exceeds the risk cap
            for (int i = start; i < end; i++) {
                if (assetValues[i] > riskCap) {
                    assetValues[i] = assetValues[i] * STRESS_PENALTY;
                }
            }
            return;
```

```
32          }
33
34          int mid = start + length / 2;
35          PortfolioThresholdTask left  = new PortfolioThresholdTask(assetValues, start, mid,
    riskCap);
36          PortfolioThresholdTask right = new PortfolioThresholdTask(assetValues, mid, end,
    riskCap);
37
38          left.fork();
39          right.compute();
40          left.join();
41      }
42  }
```

File 3: ForkJoinPortfolioApp.java

Generates a 10,000-element asset array, runs both tasks in a ForkJoinPool, checks that the parallel sum matches the serial sum and prints timings. It also benchmarks a 20,000,000-element array to show the effect of parallelism on large data.

Java Program:

J ForkJoinPortfolioApp.java

```
1   import java.util.Random;
2   import java.util.concurrent.ForkJoinPool;
3
4   public class ForkJoinPortfolioApp {
5
6       static double serialSum(double[] data) {
7           double sum = 0;
8           for (double v : data) {
9               sum += v;
10          }
11          return sum;
12      }
13
14      static double[] generate(int size, long seed) {
15          Random random = new Random(seed);
16          double[] data = new double[size];
17          for (int i = 0; i < size; i++) {
18              data[i] = 10 + random.nextDouble() * 990;   // 10.00 .. 1000.00
19          }
20          return data;
21      }
22
23      // Average time (ms) of serial vs parallel summation over 'runs' repetitions
24      static double[] timeBoth(double[] data, ForkJoinPool pool, int runs) {
25          double sink = 0;
26          long serialTotal = 0, parallelTotal = 0;
27          for (int r = 0; r < runs; r++) {
28              long t0 = System.nanoTime();
```

```
            sink += serialSum(data);
            serialTotal += System.nanoTime() - t0;

            long t1 = System.nanoTime();
            sink += pool.invoke(new ParallelRiskCalculator(data, 0, data.length));
            parallelTotal += System.nanoTime() - t1;
        }
        if (sink == 42) System.out.print("");   // keep JIT from discarding the work
        return new double[] { serialTotal / 1e6 / runs, parallelTotal / 1e6 / runs };
    }

    public static void main(String[] args) {

        final int SIZE = 10_000;
        final double RISK_CAP = 800.0;

        double[] assets = generate(SIZE, 247198L);
        ForkJoinPool pool = new ForkJoinPool();

        System.out.println("========================================");
        System.out.println("  FORK/JOIN PARALLEL PORTFOLIO ENGINE");
        System.out.println("========================================");
        System.out.println("Asset array size    : " + SIZE);
        System.out.println("Available cores     : " +
Runtime.getRuntime().availableProcessors());
        System.out.println("Sequential THRESHOLD: 1000");
        System.out.println("----------------------------------------");

        // ---- Task 1: cumulative risk score (RecursiveTask) ----
        long s0 = System.nanoTime();
        double serialRisk = serialSum(assets);
        long serialNs = System.nanoTime() - s0;

        long p0 = System.nanoTime();
        double parallelRisk = pool.invoke(new ParallelRiskCalculator(assets, 0, SIZE));
        long parallelNs = System.nanoTime() - p0;

        System.out.printf("Serial risk score   : %.2f%n", serialRisk);
        System.out.printf("Parallel risk score : %.2f%n", parallelRisk);
        System.out.println("Scores match        : " + (Math.abs(serialRisk - parallelRisk) < 1e-
6));
        System.out.printf("Single-run serial   : %.4f ms%n", serialNs / 1e6);
        System.out.printf("Single-run parallel : %.4f ms (includes pool start-up)%n", parallelNs
/ 1e6);
        System.out.println("----------------------------------------");

        // ---- Task 2: stress-test penalty (RecursiveAction) ----
        int before = 0;
        for (double v : assets) if (v > RISK_CAP) before++;

        long a0 = System.nanoTime();
        pool.invoke(new PortfolioThresholdTask(assets, 0, SIZE, RISK_CAP));
        long adjustNs = System.nanoTime() - a0;

        double stressedRisk = pool.invoke(new ParallelRiskCalculator(assets, 0, SIZE));
```

```
81
82          System.out.println("Risk cap           : " + RISK_CAP);
83          System.out.println("Assets above cap    : " + before);
84          System.out.printf("Stress-test time    : %.4f ms%n", adjustNs / 1e6);
85          System.out.printf("Risk after +5%% test : %.2f%n", stressedRisk);
86          System.out.printf("Risk increase       : %.2f (%.2f%%)%n",
87                  stressedRisk - parallelRisk, (stressedRisk - parallelRisk) / parallelRisk * 100);
88          System.out.println("----------------------------------------");
89
90          // ---- Warmed-up benchmark, 10,000 elements ----
91          timeBoth(assets, pool, 200);                    // JIT warm-up
92          double[] t = timeBoth(assets, pool, 2000);
93          System.out.println("Benchmark (10,000 elements, average of 2000 runs after warm-up)");
94          System.out.printf("  Serial   : %.5f ms%n", t[0]);
95          System.out.printf("  Parallel : %.5f ms%n", t[1]);
96          System.out.printf("  Speedup  : %.2fx%n", t[0] / t[1]);
97          System.out.println("----------------------------------------");
98
99          // ---- Larger dataset to show real parallel speedup ----
100         double[] big = generate(20_000_000, 1L);
101         timeBoth(big, pool, 3);                         // warm-up
102         double[] tb = timeBoth(big, pool, 10);
103         System.out.println("Benchmark (20,000,000 elements, average of 10 runs)");
104         System.out.printf("  Serial   : %.3f ms%n", tb[0]);
105         System.out.printf("  Parallel : %.3f ms%n", tb[1]);
106         System.out.printf("  Speedup  : %.2fx%n", tb[0] / tb[1]);
107         System.out.println("========================================");
108
109         pool.shutdown();
110     }
111 }
```

Program Output:

```
> java ForkJoinPortfolioApp
========================================
  FORK/JOIN PARALLEL PORTFOLIO ENGINE
========================================
Asset array size    : 10000
Available cores     : 1
Sequential THRESHOLD: 1000
----------------------------------------
Serial risk score   : 4993155.46
Parallel risk score : 4993155.46
Scores match        : true
Single-run serial   : 0.1236 ms
Single-run parallel : 1.7062 ms (includes pool start-up)
----------------------------------------
Risk cap            : 800.0
Assets above cap    : 1980
Stress-test time    : 0.7064 ms
Risk after +5% test : 5082302.86
Risk increase       : 89147.40 (1.79%)
----------------------------------------
Benchmark (10,000 elements, average of 2000 runs after warm-up)
  Serial   : 0.01523 ms
  Parallel : 0.04335 ms
  Speedup  : 0.35x
----------------------------------------
Benchmark (20,000,000 elements, average of 10 runs)
  Serial   : 28.265 ms
  Parallel : 33.010 ms
  Speedup  : 0.86x
========================================
```

Analysis of Timing Results

The serial and parallel risk scores are identical, which confirms that the decomposition is correct. Fork/Join speedup depends on the work per task and the number of CPU cores. With only 10,000 elements, task creation and scheduling overhead outweighs the gain, so the benchmark was repeated on a much larger array. The run reported only one available core, which is why no speedup appears in this output.

Question No. 5: Parallel IoT Sensor Search Engine Using an Abstract Class

This is an extensible parallel search engine built on an abstract parent class. Two concrete engines find the minimum temperature and the peak vibration spike using several worker threads, and the monitor application runs both searches at the same time over a 50,000-element array.

File 1: SensorSearchEngine.java

An abstract class with protected sensorData, startIdx and endIdx fields and an abstract processGrid() method that every concrete engine must implement. Main concept: abstraction and extensibility, so new engines can be added without changing the monitor.

Java Program:

SensorSearchEngine.java

```java
public abstract class SensorSearchEngine {

    protected double[] sensorData;
    protected int startIdx;
    protected int endIdx;      // exclusive

    public SensorSearchEngine(double[] sensorData, int startIdx, int endIdx) {
        this.sensorData = sensorData;
        this.startIdx = startIdx;
        this.endIdx = endIdx;
    }

    public abstract void processGrid() throws InterruptedException;
}
```

File 2: ParallelMinSearch.java

Overrides processGrid() by splitting the range into four segments, one worker thread each. Every worker finds a local minimum, and after join() the local results are reduced to the global minimum.

Java Program:

ParallelMinSearch.java

```java
public class ParallelMinSearch extends SensorSearchEngine {

```

```
3         private final int workerCount;
4         private double globalMin = Double.MAX_VALUE;
5
6         public ParallelMinSearch(double[] sensorData, int startIdx, int endIdx, int workerCount) {
7             super(sensorData, startIdx, endIdx);
8             this.workerCount = workerCount;
9         }
10
11        @Override
12        public void processGrid() throws InterruptedException {
13            double[] localMin = new double[workerCount];
14            Thread[] workers = new Thread[workerCount];
15            int total = endIdx - startIdx;
16            int chunk = total / workerCount;
17
18            for (int w = 0; w < workerCount; w++) {
19                final int id = w;
20                final int from = startIdx + w * chunk;
21                final int to = (w == workerCount - 1) ? endIdx : from + chunk;
22
23                workers[w] = new Thread(() -> {
24                    double min = Double.MAX_VALUE;
25                    for (int i = from; i < to; i++) {
26                        if (sensorData[i] < min) {
27                            min = sensorData[i];
28                        }
29                    }
30                    localMin[id] = min;
31                    System.out.printf("[MIN-Worker-%d] segment [%d..%d) local minimum = %.2f%n",
32                            id + 1, from, to, min);
33                }, "MIN-Worker-" + (w + 1));
34                workers[w].start();
35            }
36
37            for (Thread t : workers) {
38                t.join();
39            }
40
41            // Reduction of the partial results
42            for (double m : localMin) {
43                if (m < globalMin) {
44                    globalMin = m;
45                }
46            }
47        }
48
49        public double getGlobalMin() {
50            return globalMin;
51        }
52    }
```

File 3: ParallelMaxSearch.java

Works the same way as the minimum search but tracks the largest value in each segment and reduces the four local peaks to the global maximum.

Java Program:

ParallelMaxSearch.java

```java
public class ParallelMaxSearch extends SensorSearchEngine {

    private final int workerCount;
    private double globalMax = -Double.MAX_VALUE;

    public ParallelMaxSearch(double[] sensorData, int startIdx, int endIdx, int workerCount) {
        super(sensorData, startIdx, endIdx);
        this.workerCount = workerCount;
    }

    @Override
    public void processGrid() throws InterruptedException {
        double[] localMax = new double[workerCount];
        Thread[] workers = new Thread[workerCount];
        int total = endIdx - startIdx;
        int chunk = total / workerCount;

        for (int w = 0; w < workerCount; w++) {
            final int id = w;
            final int from = startIdx + w * chunk;
            final int to = (w == workerCount - 1) ? endIdx : from + chunk;

            workers[w] = new Thread(() -> {
                double max = -Double.MAX_VALUE;
                for (int i = from; i < to; i++) {
                    if (sensorData[i] > max) {
                        max = sensorData[i];
                    }
                }
                localMax[id] = max;
                System.out.printf("[MAX-Worker-%d] segment [%d..%d) local peak    = %.2f%n",
                        id + 1, from, to, max);
            }, "MAX-Worker-" + (w + 1));
            workers[w].start();
        }

        for (Thread t : workers) {
            t.join();
        }

        for (double m : localMax) {
            if (m > globalMax) {
                globalMax = m;
            }
        }
    }

    public double getGlobalMax() {
        return globalMax;
    }

}
```

File 4: SensorGridMonitorApp.java

Builds a 50,000-element telemetry array with a known minimum and maximum planted for verification, runs the min and max engines simultaneously with four workers each, prints the global values, checks them against a serial scan and logs timings. Main concept: parallel reduction.

Java Program:

J SensorGridMonitorApp.java

```java
import java.util.Random;

public class SensorGridMonitorApp {

    public static void main(String[] args) throws InterruptedException {

        final int SIZE = 50_000;
        final int WORKERS = 4;

        // Synthetic telemetry readings between 20.0 and 120.0
        Random random = new Random(247198L);
        double[] sensorData = new double[SIZE];
        for (int i = 0; i < SIZE; i++) {
            sensorData[i] = 20 + random.nextDouble() * 100;
        }

        // Plant known extremes so the result can be verified
        sensorData[12_345] = 3.75;     // critical low temperature
        sensorData[41_999] = 187.60;   // peak vibration spike

        ParallelMinSearch minSearch = new ParallelMinSearch(sensorData, 0, SIZE, WORKERS);
        ParallelMaxSearch maxSearch = new ParallelMaxSearch(sensorData, 0, SIZE, WORKERS);

        System.out.println("========================================");
        System.out.println("     IoT SENSOR GRID MONITOR");
        System.out.println("========================================");
        System.out.println("Sensor readings : " + SIZE);
        System.out.println("Workers per scan: " + WORKERS);
        System.out.println("----------------------------------------");

        // Run the MIN and MAX reductions simultaneously
        Thread minRunner = new Thread(() -> {
            try { minSearch.processGrid(); } catch (InterruptedException e) { }
        }, "Min-Engine");
        Thread maxRunner = new Thread(() -> {
```

```
            try { maxSearch.processGrid(); } catch (InterruptedException e) { }
        }, "Max-Engine");

        long start = System.nanoTime();
        minRunner.start();
        maxRunner.start();
        minRunner.join();
        maxRunner.join();
        long parallelNs = System.nanoTime() - start;

        System.out.println("----------------------------------------");
        System.out.printf("Global critical MIN temperature : %.2f%n", minSearch.getGlobalMin());
        System.out.printf("Global critical MAX vibration   : %.2f%n", maxSearch.getGlobalMax());
        System.out.println("----------------------------------------");

        // Serial check for correctness and timing comparison
        long s0 = System.nanoTime();
        double sMin = Double.MAX_VALUE, sMax = -Double.MAX_VALUE;
        for (double v : sensorData) {
            if (v < sMin) sMin = v;
            if (v > sMax) sMax = v;
        }
        long serialNs = System.nanoTime() - s0;

        System.out.println("Verification vs serial scan     : "
                + ((sMin == minSearch.getGlobalMin() && sMax == maxSearch.getGlobalMax())
                        ? "PASSED" : "FAILED"));
        System.out.printf("Parallel execution time : %.3f ms%n", parallelNs / 1e6);
        System.out.printf("Serial execution time   : %.3f ms%n", serialNs / 1e6);
        System.out.println("(Thread start-up cost dominates for only 50,000 elements)");
        System.out.println("========================================");
    }
}
```

Program Output:

```
> java SensorGridMonitorApp
========================================
     IoT SENSOR GRID MONITOR
========================================
Sensor readings : 50000
Workers per scan: 4
----------------------------------------
[MIN-Worker-1] segment [0..12500) local minimum = 3.75
[MIN-Worker-2] segment [12500..25000) local minimum = 20.02
[MIN-Worker-4] segment [37500..50000) local minimum = 20.00
[MIN-Worker-3] segment [25000..37500) local minimum = 20.02
[MAX-Worker-1] segment [0..12500) local peak    = 120.00
[MAX-Worker-3] segment [25000..37500) local peak    = 119.99
[MAX-Worker-2] segment [12500..25000) local peak    = 120.00
[MAX-Worker-4] segment [37500..50000) local peak    = 187.60
----------------------------------------
Global critical MIN temperature : 3.75
Global critical MAX vibration   : 187.60
----------------------------------------
Verification vs serial scan     : PASSED
Parallel execution time : 58.640 ms
Serial execution time   : 1.445 ms
(Thread start-up cost dominates for only 50,000 elements)
========================================
```

Explanation

The planted values, 3.75 in the first segment and 187.60 in the fourth, were correctly found as the global minimum and maximum. For an array of only 50,000 elements, the cost of creating threads exceeds the cost of scanning, so the parallel version is slower than the serial scan. Parallel reductions pay off with much larger datasets.

Question No. 6: Inventory Catalog Concurrency Benchmark

This benchmark compares two thread-safe maps holding a shared product stock catalog. One uses Collections.synchronizedMap() around a HashMap, and the other uses ConcurrentHashMap. Each runs a pool of 10 worker threads performing 50,000 lookups and updates in total, then reports execution time and throughput.

File 1: InventoryCatalogManager.java

Creates a catalog of 1,000 products with an initial stock. benchmarkSynchronizedMap() relies on coarse-grained locking (one lock for the whole map), while benchmarkConcurrentMap() relies on the fine-grained, bucket-level locking of ConcurrentHashMap. Each benchmark measures time with System.nanoTime() and verifies that no update was lost. Main concept: lock contention.

Java Program:

J InventoryCatalogManager.java

```java
import java.util.Collections;
import java.util.HashMap;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicLong;

public class InventoryCatalogManager {

    private static final int PRODUCTS       = 1000;
    private static final int INITIAL_STOCK  = 1_000_000;
    private static final int WORKER_THREADS = 10;
    private static final int TOTAL_OPS      = 50_000;
    private static final int OPS_PER_THREAD = TOTAL_OPS / WORKER_THREADS;

    // Populate a catalog: productId -> stockCount
    private static void fill(Map<String, Integer> catalog) {
        for (int i = 0; i < PRODUCTS; i++) {
            catalog.put("PROD-" + i, INITIAL_STOCK);
        }
    }

    // Runs 10 workers; each does lookups and stock updates. Returns elapsed nanoseconds.
    private static long runWorkload(Map<String, Integer> catalog, AtomicLong updates)
            throws InterruptedException {
```

```
28
29          ExecutorService pool = Executors.newFixedThreadPool(WORKER_THREADS);
30          long start = System.nanoTime();
31
32          for (int t = 0; t < WORKER_THREADS; t++) {
33              final int seed = t;
34              pool.submit(() -> {
35                  java.util.Random rnd = new java.util.Random(seed);
36                  for (int op = 0; op < OPS_PER_THREAD; op++) {
37                      String id = "PROD-" + rnd.nextInt(PRODUCTS);
38                      if (op % 2 == 0) {
39                          catalog.get(id);                        // stock lookup
40                      } else {
41                          catalog.merge(id, -1, Integer::sum);    // atomic stock update
42                          updates.incrementAndGet();
43                      }
44                  }
45              });
46          }
47
48          pool.shutdown();
49          pool.awaitTermination(1, TimeUnit.MINUTES);
50          return System.nanoTime() - start;
51      }
52
53      private static long totalStock(Map<String, Integer> catalog) {
54          long sum = 0;
55          for (int v : catalog.values()) sum += v;
56          return sum;
57      }
58
59      private static void report(String name, Map<String, Integer> catalog,
60                                 long nanos, long updates) {
61          double ms = nanos / 1e6;
62          double throughput = TOTAL_OPS / (nanos / 1e9);
63          long expected = (long) PRODUCTS * INITIAL_STOCK - updates;
64          System.out.println("Collection     : " + name);
65          System.out.printf("Execution time : %.3f ms%n", ms);
66          System.out.printf("Throughput     : %,.0f operations/second%n", throughput);
67          System.out.println("Data integrity : "
68                  + (totalStock(catalog) == expected ? "PASSED (no lost updates)" : "FAILED"));
69      }
70
71      public static void benchmarkSynchronizedMap() throws InterruptedException {
72          Map<String, Integer> catalog = Collections.synchronizedMap(new HashMap<>());
73          fill(catalog);
74          AtomicLong updates = new AtomicLong();
75          long nanos = runWorkload(catalog, updates);
76          report("Collections.synchronizedMap(HashMap)", catalog, nanos, updates.get());
77      }
78
79      public static void benchmarkConcurrentMap() throws InterruptedException {
80          Map<String, Integer> catalog = new ConcurrentHashMap<>();
81          fill(catalog);
82          AtomicLong updates = new AtomicLong();
```

```
83            long nanos = runWorkload(catalog, updates);
84            report("ConcurrentHashMap", catalog, nanos, updates.get());
85        }
86
87        public static void main(String[] args) throws InterruptedException {
88
89            // Warm-up so the JIT compiler does not distort the first measurement
90            for (int i = 0; i < 5; i++) {
91                runWorkload(Collections.synchronizedMap(prefilled()), new AtomicLong());
92                runWorkload(prefilled(new ConcurrentHashMap<>()), new AtomicLong());
93            }
94
95            System.out.println("==========================================");
96            System.out.println("  INVENTORY CATALOG CONCURRENCY BENCHMARK");
97            System.out.println("==========================================");
98            System.out.println("Products       : " + PRODUCTS);
99            System.out.println("Worker threads : " + WORKER_THREADS);
100           System.out.println("Total ops      : " + TOTAL_OPS + " (lookups + updates)");
101           System.out.println("------------------------------------------");
102
103           benchmarkSynchronizedMap();
104           System.out.println("------------------------------------------");
105           benchmarkConcurrentMap();
106           System.out.println("==========================================");
107       }
108
109       private static Map<String, Integer> prefilled() {
110           return prefilled(new HashMap<>());
111       }
112
113       private static Map<String, Integer> prefilled(Map<String, Integer> map) {
114           fill(map);
115           return map;
116       }
117   }
```

Program Output:

```
> java InventoryCatalogManager
==========================================
  INVENTORY CATALOG CONCURRENCY BENCHMARK
==========================================
Products       : 1000
Worker threads : 10
Total ops      : 50000 (lookups + updates)
------------------------------------------
Collection     : Collections.synchronizedMap(HashMap)
Execution time : 14.362 ms
Throughput     : 3,481,460 operations/second
Data integrity : PASSED (no lost updates)
------------------------------------------
Collection     : ConcurrentHashMap
Execution time : 11.818 ms
Throughput     : 4,230,926 operations/second
Data integrity : PASSED (no lost updates)
==========================================
```

Explanation

Collections.synchronizedMap() guards every operation with a single mutex, so all 10 threads queue for the same lock. ConcurrentHashMap reads without locking and locks only the bucket being updated, so threads working on different products do not block one another. A warm-up phase runs first so JIT compilation does not distort the first measurement, and the integrity check shows both maps handled every update correctly. The gap in throughput widens as cores and threads increase.

Question No. 7: Cache Locality Benchmark: Row-Major vs Column-Major Traversal

This benchmark builds a 2000 × 2000 double matrix and sums it in two ways: with the loops ordered i then j (row-major) and with them swapped to j then i (column-major). Each traversal is timed over 10 iterations, and the speedup ratio is explained through CPU cache architecture and spatial locality.

File 1: CacheLocalityBenchmark.java

traverseRowMajor() walks along a row, which is contiguous memory, while traverseColumnMajor() steps down a column, touching a different row array on every step. The main method fills the matrix, warms up the JIT, runs 10 timed iterations and prints each time in milliseconds, the averages and the ratio. Main concept: spatial locality and cache lines.

Java Program:

J CacheLocalityBenchmark.java

```java
public class CacheLocalityBenchmark {

    private static final int N = 2000;
    private static final int ITERATIONS = 10;

    private static double[][] matrix = new double[N][N];

    // Row-major: inner loop walks along a row (contiguous memory, stride 1)
    public static double traverseRowMajor() {
        double sum = 0;
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++) {
                sum += matrix[i][j];
            }
        }
        return sum;
    }

    // Column-major: inner loop jumps down a column (a different row array every step)
    public static double traverseColumnMajor() {
        double sum = 0;
        for (int j = 0; j < N; j++) {
            for (int i = 0; i < N; i++) {
                sum += matrix[i][j];
            }
        }
        return sum;
```

```java
    }

    public static void main(String[] args) {

        // Fill the 2000 x 2000 matrix
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++) {
                matrix[i][j] = (i * 31 + j * 17) % 100 / 10.0;
            }
        }

        // JIT warm-up (not timed)
        double checksum = 0;
        for (int w = 0; w < 3; w++) {
            checksum += traverseRowMajor();
            checksum += traverseColumnMajor();
        }

        double[] rowTimes = new double[ITERATIONS];
        double[] colTimes = new double[ITERATIONS];
        double rowTotal = 0, colTotal = 0;

        System.out.println("========================================");
        System.out.println("     CACHE LOCALITY BENCHMARK (2000 x 2000)");
        System.out.println("========================================");
        System.out.println("Iteration | Row-Major (ms) | Column-Major (ms)");
        System.out.println("----------------------------------------");

        for (int it = 0; it < ITERATIONS; it++) {
            long t0 = System.nanoTime();
            double r = traverseRowMajor();
            long t1 = System.nanoTime();
            double c = traverseColumnMajor();
            long t2 = System.nanoTime();

            checksum += r + c;
            rowTimes[it] = (t1 - t0) / 1e6;
            colTimes[it] = (t2 - t1) / 1e6;
            rowTotal += rowTimes[it];
            colTotal += colTimes[it];

            System.out.printf("%9d | %14.3f | %17.3f%n", it + 1, rowTimes[it], colTimes[it]);
        }

        double rowAvg = rowTotal / ITERATIONS;
        double colAvg = colTotal / ITERATIONS;

        System.out.println("----------------------------------------");
        System.out.printf("Average Row-Major time    : %.3f ms%n", rowAvg);
        System.out.printf("Average Column-Major time : %.3f ms%n", colAvg);
        System.out.printf("Speedup (Column / Row)    : %.2fx%n", colAvg / rowAvg);
        System.out.println("Checksum (prevents dead-code removal): " + checksum);
        System.out.println("========================================");
    }
}
```

Program Output:

```
> java CacheLocalityBenchmark
========================================
     CACHE LOCALITY BENCHMARK (2000 x 2000)
========================================
Iteration | Row-Major (ms) | Column-Major (ms)
----------------------------------------
        1 |          5.899 |            45.097
        2 |          6.304 |            45.855
        3 |          5.727 |            44.055
        4 |          5.794 |            43.778
        5 |          5.797 |            46.946
        6 |          8.375 |            49.361
        7 |          5.665 |            45.292
        8 |          5.684 |            44.392
        9 |          6.125 |            44.177
       10 |          5.690 |            44.335
----------------------------------------
Average Row-Major time    : 6.106 ms
Average Column-Major time : 45.329 ms
Speedup (Column / Row)    : 7.42x
Checksum (prevents dead-code removal): 5.148E8
========================================
```

Execution Report: How Cache Architecture Explains the Results

Modern CPUs do not fetch single bytes from RAM; they load a whole cache line, typically 64 bytes, into the L1 and L2 caches. A double takes 8 bytes, so one cache line holds 8 consecutive doubles.

Row-major traversal reads neighbouring elements (stride 1). A single cache miss loads a line that serves the next 7 reads as hits, and the hardware prefetcher recognises the sequential pattern and loads upcoming lines early, giving a high hit rate.

Column-major traversal reads elements that sit about 16 KB apart, because each Java row is a separate array of 2000 doubles. Every read touches a different cache line, and that line is usually evicted before its other 7 elements are needed on the next column pass. Nearly every access is therefore a miss, the prefetcher cannot predict the jumping pattern, and the CPU stalls waiting for main memory and for TLB lookups across many pages.

Both loops perform exactly the same number of additions, so the difference comes purely from memory access order. The measured ratio (column-major time divided by row-major time) was 7.42x, close to the theoretical figure of about 8 accesses served per cache line. The exact value depends on the processor, cache sizes and memory speed.